

Compel–ZipMix: Compression-Aligned Mixtures for Efficient Task-Aware Language-Model Pre-training

Brando Miranda^{1*} Elyas Obbad^{1*} Sanmi Koyejo¹

¹Stanford University

{bmiranda, eobbad, sanmi}@stanford.edu

April 15, 2026

Abstract

Mixture composition is a pivotal yet under-explored axis of large-language-model pre-training. DOREMI learns domain weights via Group-DRO, but empirical replications report accuracy losses: its coarse, hand-defined domains blur signal and noise, and its worst-case objective begins from an uninformative uniform prior—two assumptions at odds with the No-Free-Lunch theorem wolpert1997no. We present *Compel-ZipMix*, which partitions the corpus into compression-ratio buckets and assigns each bucket a weight derived from a ZIP-based similarity to a small “AGI” validation suite. ZipMix includes a zero-cost *Static* variant that samples directly from this alignment prior and a *DRO* variant that refines the prior with a single Group-DRO pass. Using an X-parameter proxy on THE PILE, ZipMix-Static outperforms a uniform mix by **Y pp**, while ZipMix-DRO reaches the same score **X**× **faster**; both mixtures transfer unchanged to ×10 and ×100 models, remaining below the three-order “dark-matter” scaling limit in the GPT-4 report. Because the prior depends solely on the chosen validation set, a single CPU-only ZIP-FIT pass is enough to retarget the mixture toward any new task suite, enabling *practically free, domain-conditioned* foundation checkpoints such as MMLU+MINIF2F or MMLU+FINEWEB+PROOFNET+MATH.

1 Introduction

The quality of a pre-training *mixture* is as influential as token count and parameter size. When high-entropy crawl dominates, large language models (LLMs) devote compute to memorising noise, and downstream performance stalls.

DOREMI tackles mixture design with distributionally robust optimisation (Group-DRO). While its original study reported speed-ups, independent attempts often observe

*Equal contribution.

that DoReMi funnels probability mass into noisy domains and can reduce accuracy fan2023doge. We trace this outcome to two linked assumptions:

H_{prior} — *No task prior.* DoReMi starts from a uniform weighting of coarse domains (*Wikipedia, Books, ...*). By the No-Free-Lunch theorem wolpert1997no, a learner with no prior preference is free to prefer entropy; the optimiser thus boosts whichever bucket maximises loss—often noise.

H_{max} — *Over-pessimistic objective.* The inner maximisation always targets the highest excess-loss domain, conflating “hard because useful” with “hard because irreducible” and systematically over-weighting diversity.

Compel-ZipMix. To test and remedy these hypotheses we introduce two steps:

1. *Compression buckets* let the corpus self-partition by compression ratio, a model-free proxy for information density.

2. *Alignment prior* assigns each bucket a weight $\alpha_m^{(0)} \propto \sum_{d \in B_m} s(d) + \tau$ with $s(d) = 1 - \text{NCD}_{\text{zip}}(d, \mathcal{V})$ and $\tau = 10^{-3}$, where $\mathcal{V}$ is a small validation suite (MMLU, GSM8K, MiniF2F, ...).

ZipMix-Static samples directly from $\alpha^{(0)}$ (addressing H_{prior}); *ZipMix-DRO* adds one Group-DRO refinement (tempering H_{max}). We cap model size jumps to 10^3 to avoid the empirical “dark-matter” deviation of scaling laws gpt4report,schaeffer2024predictive.

Empirical preview. On THE PILE and FINEWEB a proxy of X parameters trained with ZipMix-Static exceeds the uniform baseline by +Y pp on an AGI suite; ZipMix-DRO adds Y pp and reaches baseline accuracy X× faster. The mixture transfers unchanged to models ×10 and ×100 larger.

Free, task-conditioned checkpoints. Because $\alpha^{(0)}$ depends only on the validation corpus, re-scoring the data with a new suite—say MMLU+MINIF2F for maths or MMLU+FINEWEB+PROOFNET+MATH+PUTNAM-AXIOM for formal reasoning—requires a single ZIP-FIT sweep and an optional short proxy refresh, giving any lab the ability to mint *domain-specialised yet broadly capable* foundation models on commodity hardware.

Contributions.

- We convert DoReMi’s rumoured failure into two explicit hypotheses and validate them empirically.
- We introduce compression buckets plus alignment priors, yielding ZipMix-Static and ZipMix-DRO—cheap, robust, and task-aware.
- We demonstrate X× faster convergence and +Y pp accuracy gains, and release code, bucket assignments, and scoring scripts for reproducibility.

2 Method

2.1 Notation

Let $\mathcal{D} = \{d_1, \dots, d_N\}$ be the unlabeled corpus and $\mathcal{V}$ a small validation suite whose tasks define the desired prior. After the compression–ratio split (Sec. ??) we obtain a

set of buckets

$$\mathcal{B} = \{B_1, \dots, B_{|\mathcal{B}|}\},$$

where $|\mathcal{B}|$ denotes the number of buckets. We write $\ell_\theta(x)$ for the per-token negative log-likelihood of a language model with parameters θ and use $\text{NCD}_{\text{zip}}(\cdot, \cdot)$ for the normalised compression distance.

2.2 Step 1: Compression buckets

For each document $d \in \mathcal{D}$ compute the compression ratio

$$\text{CR}(d) = \frac{\text{bytes}_{\text{zip}}(d)}{\text{bytes}_{\text{raw}}(d)}. \quad (1)$$

A sequence of thresholds $0 = c_0 < c_1 < \dots < c_{|\mathcal{B}|} = 1$ induces the buckets $B_m = \{d: \text{CR}(d) \in [c_{m-1}, c_m)\}$.

2.3 Step 2: Alignment prior

Define the ZIP-FIT similarity

$$s(d) = 1 - \text{NCD}_{\text{zip}}(d, \mathcal{V}).$$

The alignment weight for bucket m is

$$\alpha_m^{(0)} = \frac{\sum_{d \in B_m} s(d) + \tau}{\sum_{B_j \in \mathcal{B}} \sum_{d \in B_j} s(d) + |\mathcal{B}| \tau}, \quad \tau = 10^{-3}, \quad (2)$$

yielding $\alpha^{(0)} \in \Delta^{|\mathcal{B}|-1}$.

2.4 Idea 0: Alignment-Weighted Pre-Training (AWPT)

Training minimises

$$\theta^* = \arg \min_{\theta \in \Theta} \sum_{B_m \in \mathcal{B}} \alpha_m^{(0)} \frac{1}{|B_m|} \sum_{x \in B_m} \ell_\theta(x). \quad (3)$$

Stochastic optimisation uses hierarchical sampling: sample $m \sim \text{Categorical}(\alpha^{(0)})$, then $x \sim \text{Uniform}(B_m)$.

2.5 Idea 1: Loss-Reweighted AWPT

Keep the dataloader uniform over $\mathcal{D}$ and scale each loss term by $\alpha_{m(x)}^{(0)}$, where $m(x)$ maps a document to its bucket.

2.6 Idea 2: ZipMix-DRO (One-Shot Refinement)

Starting from $\alpha^{(0)}$, run one Group-DRO pass on a proxy model to obtain α^* (Alg. ??) and then optimise Eq. (??) with α^* .

Algorithm 1 ZipMix-DRO: One-Shot Group-DRO Refinement

Require: Buckets $\mathcal{B}$, alignment prior $\alpha^{(0)}$, proxy model θ_0 , step size η , steps T

Ensure: Refined weights α^*

```
1:  $\alpha \leftarrow \alpha^{(0)}$ 
2: for  $t = 1, \dots, T$  do
3:   Sample batch  $x \sim \text{hierarchical sampler}(\alpha)$ 
4:   Update  $\theta$  with gradient step on  $\ell_\theta(x)$ 
5:    $\hat{\ell}_m \leftarrow \text{average loss on bucket } B_m$   $\triangleright$  excess loss
6:    $\alpha_m \leftarrow \alpha_m \exp(\eta \hat{\ell}_m)$ ; normalise  $\alpha$ 
7: end for
8: return  $\alpha^* \leftarrow \alpha$ 
```

2.7 Scaling constraint

We limit the target model to at most $10^3 \times$ the proxy size, remaining below the empirical “dark-matter” regime where scaling laws deviate gpt4report,schaeffer2024predictive.

3 Experiments

Setup. Proxy size X ; target sizes $X \times 10$ and $X \times 100$. Datasets: THE PILE and FINEWEB. Validation suite: MMLU-dev, GSM8K-dev, MiniF2F-dev.

Main results. ZipMix-Static beats uniform by $+X$ pp. ZipMix-DRO gains an extra $+Y$ pp and reaches baseline accuracy $X \times$ sooner.

Ablations. Without the alignment prior, optimisation collapses into the noisiest bucket; replacing CR buckets with hand domains reproduces DoReMi’s failure.

4 Dark-Matter Scaling

Scaling laws fit a power curve up to $\approx 10^3$ scale; beyond that the loss deviates unpredictably, a phenomenon termed *dark matter* schaeffer2024predictive. All ZipMix evaluations remain below this threshold.

5 Discussion and Limitations

Our method depends on the relevance of the chosen validation suite and on the language neutrality of compression ratio. Multilingual corpora and radically different downstream tasks may require re-scoring. Group-DRO adds $X\%$ extra proxy compute; ZipMix-Static offers a cheaper fallback.